

Password Reset & Recovery Workflow Specification

Anti-Enumeration Neutrality, PKCE Recovery Code Exchange, NIST SP 800-63B Validation & Global Session Termination

Jira Reference: SCRUM-59

Author: Joan Marie Encallado Inting (Engineering Team)

GitHub Repository Source: github.com/hermarDev/Abang_Cebu_AI_Group2/blob/main/docs/auth-password-reset-spec.md

Framework: Next.js 16 (App Router) + Supabase Auth (@supabase/ssr)

Reviewed By: Hermar Centillas (Lead / Scrum Master)

Status: APPROVED / DONE

Sprint: Sprint 1 (Architecture & Governance)

1. Purpose & Threat Context

Account recovery is a prime target for attack vectors in real estate platforms. Compromising a landlord's account enables unauthorized alteration of payment credentials and fraudulent deposit theft, while compromising a renter's account exposes private Philippine identification records and active lease inquiries.

AbangCebu AI establishes a **defense-in-depth password recovery pipeline** combining anti-enumeration generic responses, rolling rate limits (3 requests per 15 minutes), single-use 60-minute cryptographically signed PKCE recovery codes, NIST SP 800-63B password complexity enforcement, and mandatory global session termination (`scope: 'global'`) across all devices upon successful update.

Zero-Trust Account Takeover Defense: When a user updates their password, all existing sessions and refresh token families are immediately destroyed. If an adversary holds a hijacked token, they are immediately expelled platform-wide.

2. Password Recovery State Machine

Current State	Trigger / Event	Action Taken	Next State
UNAUTHENTICATED	User inputs email on /forgot-password	Collect <code>PasswordResetRequestPayload</code>	REQUESTING_RESET
REQUESTING_RESET	POST /api/auth/reset-password/ request	Check 15-min rate limit & Turnstile; trigger GoTrue reset; emit generic 200 OK	TOKEN_DISPATCHED
TOKEN_DISPATCHED	Supabase sends email with PKCE link	Email delivered with 1-hour single-use authorization code	EXCHANGING_CODE
EXCHANGING_CODE	User clicks link; browser reaches /auth/callback	Server executes <code>exchangeCodeForSession(code)</code> ; sets recovery cookie	RECOVERY_SESSION_ACTIVE
RECOVERY_SESSION_ACTIVE	Redirect to /reset-password	Render reset form; validate new password strength	UPDATING_PASSWORD
UPDATING_PASSWORD	POST /api/auth/reset-password/ confirm	Execute <code>supabase.auth.updateUser({ password })</code>	GLOBAL_REVOCATION
GLOBAL_REVOCATION	Password update succeeds	Purge all rows in <code>auth.sessions</code> ; broadcast <code>SIGNED_OUT</code> to tabs	UNAUTHENTICATED_REDIRECT

3. Anti-Enumeration Firewall & Rate Limiting

To prevent malicious email harvesting, the reset request endpoint adheres strictly to anti-enumeration neutrality:

- Constant Response:** The endpoint strictly emits HTTP 200 with `"If an account is associated with this email, a password recovery link has been dispatched."` regardless of email registration status.
- Timing Attack Defense:** Artificial jitter (200ms–400ms delay) is applied so non-existent account lookups match the latency of outbound email generation.
- Rate Limiting & CAPTCHA:** Rolling limit of 3 requests per 15 minutes per email and 5 per IP, reinforced by Cloudflare Turnstile token validation.

4. PKCE Recovery Code Exchange Protocol

AbangCebu AI utilizes the PKCE authorization code exchange flow rather than legacy implicit hash fragments (`#access_token=...`) to prevent token interception in browser referrer headers.

```
// Next.js 16 Route Handler: /auth/callback
export async function GET(request: NextRequest) {
  const { searchParams, origin } = new URL(request.url);
  const code = searchParams.get('code');
  const type = searchParams.get('type');
  const next = searchParams.get('next') || '/reset-password';

  if (!code || type !== 'recovery') {
    return NextResponse.redirect(new URL('/login?error=invalid_link', origin));
  }

  const cookieStore = await cookies();
  const supabase = createServerClient(SUPABASE_URL, SUPABASE_ANON_KEY, {
    cookies: {
      getAll: () => cookieStore.getAll(),
      setAll: (cookiesToSet) => {
        cookiesToSet.forEach(({ name, value, options }) => cookieStore.set(name, value, options));
      },
    },
  });

  // Exchange one-time PKCE code for scoped recovery session
  const { error } = await supabase.auth.exchangeCodeForSession(code);
  if (error) {
    return NextResponse.redirect(new URL('/login?error=expired_recovery_link', origin));
  }

  // Open-redirect defense: Sanitize target destination
  const destination = isSafeRedirectUrl(next) ? next : '/reset-password';
  return NextResponse.redirect(new URL(destination, origin));
}
```

5. NIST SP 800-63B Password Validation Rules

New passwords must pass strict verification prior to database commitment:

- **Length:** 8 to 72 characters (preventing DoS attacks against cryptographic hashing algorithms).
- **Character Diversity:** Minimum 3 of 4 classes: uppercase (A-Z), lowercase (a-z), digits (0-9), and symbols (!@#\$\$%^&*...).
- **Disparity Match:** Server-side validation enforcing `newPassword !== confirmPassword`.

6. Mandatory Global Session Revocation

Upon password update, the server executes `supabase.auth.signOut({ scope: 'global' })`. This triggers:

- **Database Purge:** All rows in `auth.sessions` matching the user's ID are deleted.
- **Refresh Token Invalidation:** All issued refresh token families are marked revoked.
- **Multi-Tab Sign-Out:** Client emits `SIGNED_OUT` across `BroadcastChannel('supabase.auth.token')`.
- **Safe Navigation:** Navigates user to `/login?message=password_reset_success` to re-authenticate with new credentials.

7. Open-Redirect Defense Matrix

Input Target Destination	Firewall Rule	Validation	Resolved Route
<code>/reset-password</code>	Valid relative root path	PASSED	<code>/reset-password</code>
<code>https://phishing-site.com</code>	External absolute URL	BLOCKED	<code>/reset-password</code>
<code>//attacker.com/malicious</code>	Protocol-relative URL	BLOCKED	<code>/reset-password</code>
<code>/\attacker.com</code>	Backslash path injection	BLOCKED	<code>/reset-password</code>

8. Verification & Definition of Done (DoD)

Audit Sign-Off: All acceptance criteria from Jira SCRUM-59 are fully satisfied. TypeScript contracts in `src/types/auth.ts` compile with 0 errors and 0 lint warnings. Zero premature React/JSX UI components were created.